PRN232 – Final Assignment

1. Objective

The objective of this assignment is to design, develop, and deploy a distributed application using the .NET ecosystem. Students are expected to apply the concepts of RESTful API, asynchronous messaging, background processing, inter-service communication, containerization, and cloud-native deployment.

---

2. Functional Requirements

Students are free to choose any application domain (e.g., e-commerce, education, healthcare, logistics, IoT, smart campus, booking, finance, etc.). The solution must include the following components:

2.1 REST API Service

- Develop at least one ASP.NET Core REST API.
- Follow RESTful design principles and naming conventions.
- Implement CRUD operations for core business entities.
- Apply layered architecture (API – Services – Repository).
- Implement authentication using JWT.
- Support searching, filtering, sorting, and pagination where appropriate.

2.2 Background Job

- Implement at least one background processing service.
- The background job should execute scheduled or asynchronous tasks, such as:
- Sending notifications or emails.
- Data synchronization.
- Report generation.
- Data cleanup.
- Inventory updates.
- Other business-related background processes.

2.3 Message Broker

- Integrate a message broker to enable asynchronous communication.
- Students may choose one of the following technologies:
- Apache Kafka
- Redis (Pub/Sub or Streams)
- Demonstrate at least one producer and one consumer.

2.4 gRPC Service

- Develop an independent service communicating through gRPC.
- Demonstrate interaction between the REST API and the gRPC service.
- Typical use cases include:
- Recommendation service.

- Pricing service.
- AI/ML inference service.
- Inventory service.
- User profile service.

---

3. Deployment Requirements

The entire system must be deployable using one of the following approaches:

- Docker Desktop with Docker Compose.
- A public cloud platform (Azure, AWS, Google Cloud, Render, Railway, etc.).

Deployment must include all required services and demonstrate successful communication among system components.

---

4. Technical Requirements

The project should demonstrate the following technical capabilities:

- ASP.NET Core (.NET 8 or later).
- Entity Framework Core.
- SQL Server or another relational database.
- Dependency Injection.
- Configuration management.
- Logging and exception handling.
- JWT Authentication.
- RESTful API design.
- gRPC communication.
- Message Broker integration.
- Background Service.
- Docker containerization.

---

5. Deliverables

Each team must submit:

- Complete source code.
- Database scripts or migrations.
- Dockerfile(s).
- Docker Compose configuration (if applicable).
- API documentation (Swagger/OpenAPI).
- README.md containing:
- Project description.
- System architecture.
- Technology stack.
- Installation guide.

- Deployment instructions.
- Team member responsibilities.

---

6. Demonstration

During the final presentation, each team must demonstrate:

- System architecture.
- REST API functionality.
- Background job execution.
- Message publishing and consuming.
- gRPC communication.
- Docker or cloud deployment.
- End-to-end business workflow.

---

7. Assessment Criteria

Criteria	Weight
System architecture and design	20%
REST API implementation	20%
Background Job	10%
Message Broker integration	15%
gRPC service	15%
Docker/Cloud deployment	10%
Documentation and presentation	10%

---

8. Notes

- Students may work individually or in teams (according to the course policy).
- Application domain is completely open; however, the solution must clearly demonstrate all required technologies.
- Additional features beyond the minimum requirements will be considered as bonus during evaluation, provided they are well designed and implemented.